

Author

Ashutosh Ray Mohapatra

24f2004660

24f2004660@ds.study.iitm.ac.in

About me: I am currently pursuing the **BS Degree in Data Science and Applications** at the **Indian Institute of Technology Madras** and a **BTech in CSE(AIML)** in **Gandhi Engineering College, Bhubaneswar**. This project was developed as part of the **Modern Application Development I** course to apply the concepts of Flask, database management, authentication, and role-based web application development.

Description

This project is a Trekking Management web app made with Flask. There are three types of users that are Admin, Staff, and regular Users. Admin can approve/reject staff and user signups, and creates treks and assigns them to staff. Staff can manage the treks assigned to them and see who's booked. Users can browse the list of treks, search/filter them, and book a trek if slots are available.

AI/LLM usage

I used Gemini (Gemini 3 Flash) to assist in structuring the project's database schema and for bootstrap syntax, the boiler plate provided we reused and manually edited. The extent of AI/LLM usage is approximately 14-16%, primarily focused on syntax optimization and generating documentation templates. All core business logic, database migrations, and debugging were done manually, with the implementation inspired by concepts demonstrated during the live project sessions.

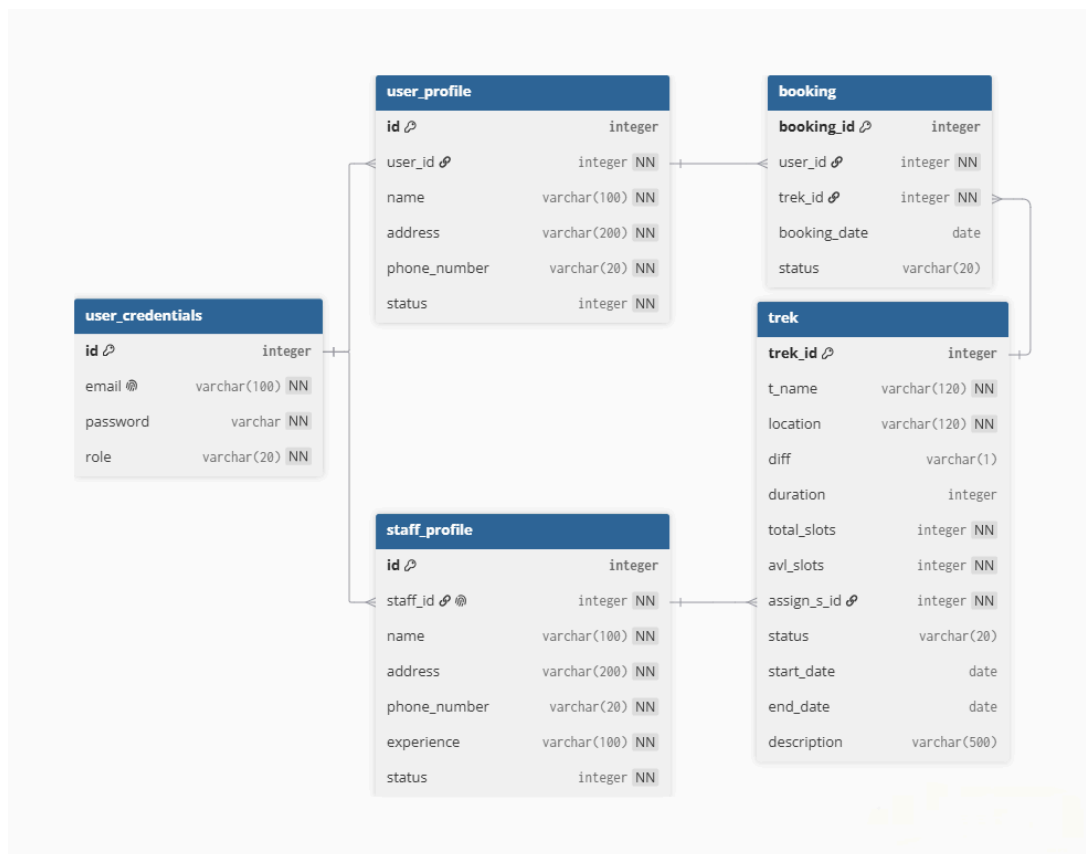
Flask	2%
Bootstrap	12%

Technologies used

<u>Technology / Library</u>	<u>Purpose</u>
Flask Core	backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design

DB Schema Design

The application uses **SQLite** as the database and **SQLAlchemy ORM** for managing data models and relationships. It consists of three main tables: **Users**, **Treks**, and **Bookings**. Staff members are stored in the **Users** table and are identified using the **role** attribute.



Tables:

1. **user_credentials** — stores login details (id, email, password, role)
2. **user_profile** — stores trekker profile details (id, user_id, name, address, phone number, status)
3. **staff_profile** — stores trek leader details (id, staff_id, name, address, phone number, experience, status)
4. **trek** — stores treks created by admin (trek_id, name, location, difficulty, duration, total slots, available slots, assigned staff id, status, start date, end date, description)
5. **booking** — stores bookings made by users for treks (booking_id, user_id, trek_id, booking date, status)

Relationships:

- One-to-One → user_credentials → user_profile
- One-to-One → user_credentials → staff_profile
- One-to-Many → staff_profile → trek
- One-to-Many → trek → booking
- One-to-Many → user_profile → booking

Architecture

The project follows an MVC-style structure.

- **app.py** — sets up the Flask app, connects it to the SQLite database, and imports the controller so routes get registered.
- **models.py** — has all the database models (User_Credentials, user_profile, staff_profile, Trek, Booking) and the shared **db** object.
- **controller.py** — has all the routes (login, register, admin/staff/user actions) in one file, along with helper functions like **search_user**, **search_staff**, **search_trek** that are reused across routes.
- **templates/** — HTML pages split by role: **Admin_templates/**, **Staff_templates/**, **User_templates/**, each with a **components/** folder holding a reusable menu sidebar. **base.html** has the shared layout (navbar, Bootstrap) that other pages extend.
- **static/** — CSS and images.

Features Implemented

Authentication & registration:

- Single registration form for all roles. Users are active right away, staff go into a pending queue for admin approval.
- Login redirects to the right dashboard based on role and approval status.

Admin features:

- Dashboard with trek, user, staff and booking counts.
- Create, edit, delete treks — set staff, slots, dates, difficulty, status.
- Manage/search treks, staff (approve/reject) and users (approve/blacklist).
- Cancel bookings.

Staff features:

- Dashboard with assigned treks, open treks, and participant counts.
- View assigned treks, manage participants, update slots/status, reset slots, mark treks completed.

User features:

- Browse/search treks by difficulty and location, see what's already booked, book open treks.
- View booking history.

Shared:

- Profile view/edit for users and staff.

Git-hub Repository link:

https://github.com/24f2004660/Trekking_Management

Video link:

<https://drive.google.com/file/d/1BAWm5tXHqfuPAFXiPG53ap3QcyR68DI/view?usp=sharing>